Laboratorio di Architetture Software e Sicurezza Informatica [AAF1569]

2 - Software Architectures

SAPIENZA
UNIVERSITÀ DI ROMA

Part of therein contents are based on slides of Proff, Massimo Mecella, John Yang and Ian Sommerville

DIAG
Dipartimento di Ingegneria informatica, automatica e gestionale Antonio Ruberti

Software Architecture

It's the "**overall organization**" of a software product:

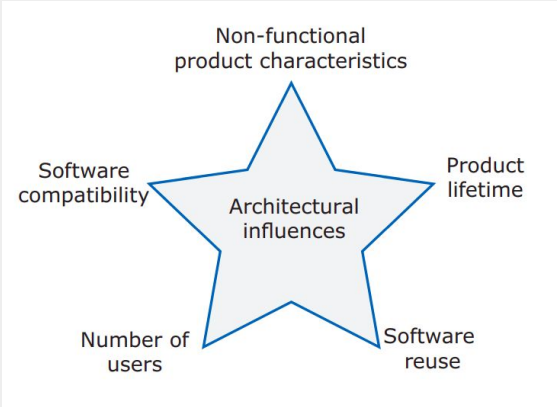
- **software components,**
- **server organization,**
- **technologies**

Impacts non-functional system quality **attributes**:

- responsiveness
- reliability
- availability
- security
- usability
- maintainability
- resilience

It is impossible to optimize all of the non-functional attributes in the same system

The software architecture affect the complexity of a product



Software Architecture

Trade-offs:

- **maintainability vs performance**
  - maintainability ⇒ many self-contained parts (easy to replace/enhance)
    ⇒ components takes time to communicate

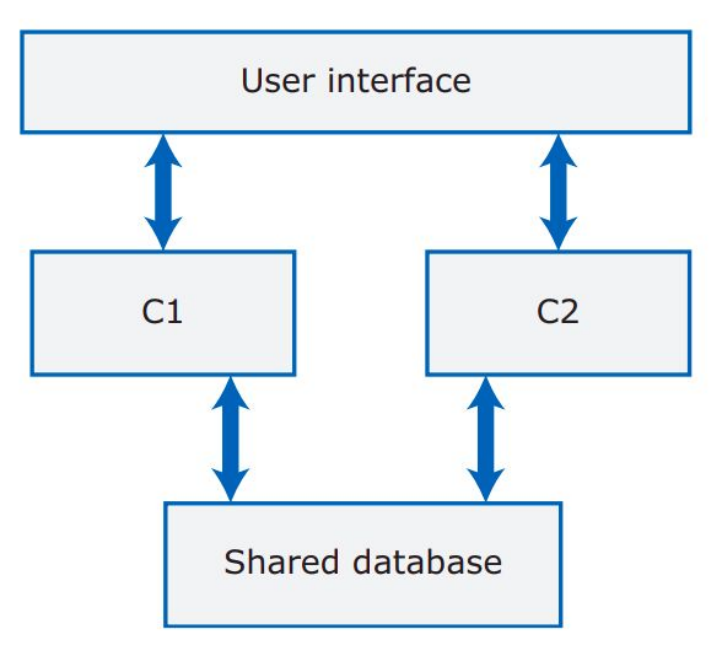
- **security vs usability**
  - multiple security layers ⇒ slower and/or more complex interaction

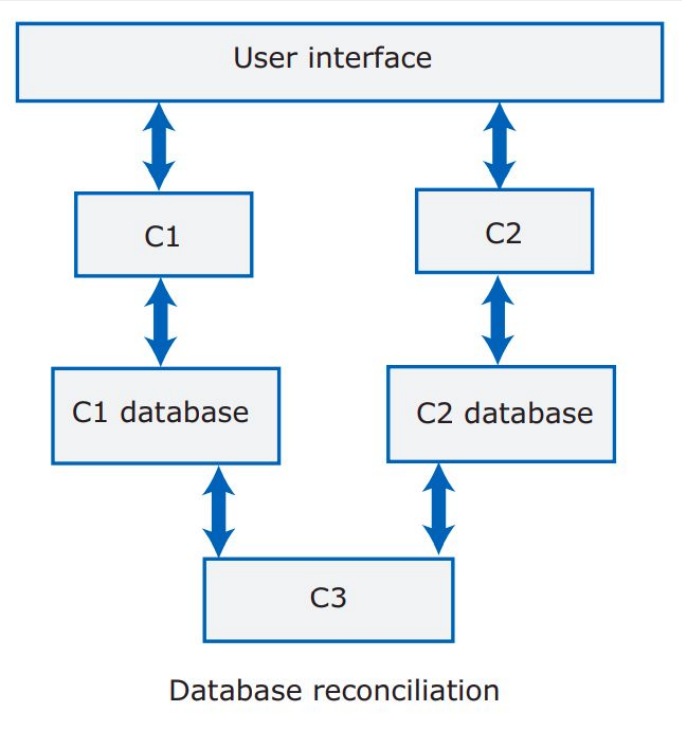
- **availability vs time to market and cost**
  - redundancy ⇒ costs and complexity

Then:

- How organize the system?
- How the component are distributed and communicate with each other?
- What technologies are used?

Architecture Examples





Database reconciliation

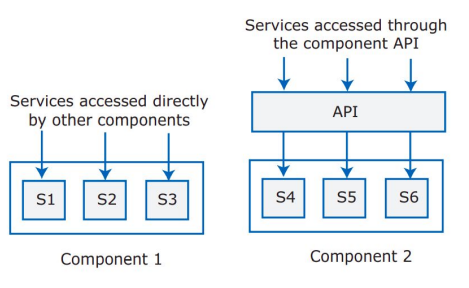
Software Components: Component, Service and Microservice

A **component** is an element that implements a **coherent set of functionality** or features, i.e. it offers one or more service, that may be used by other components.

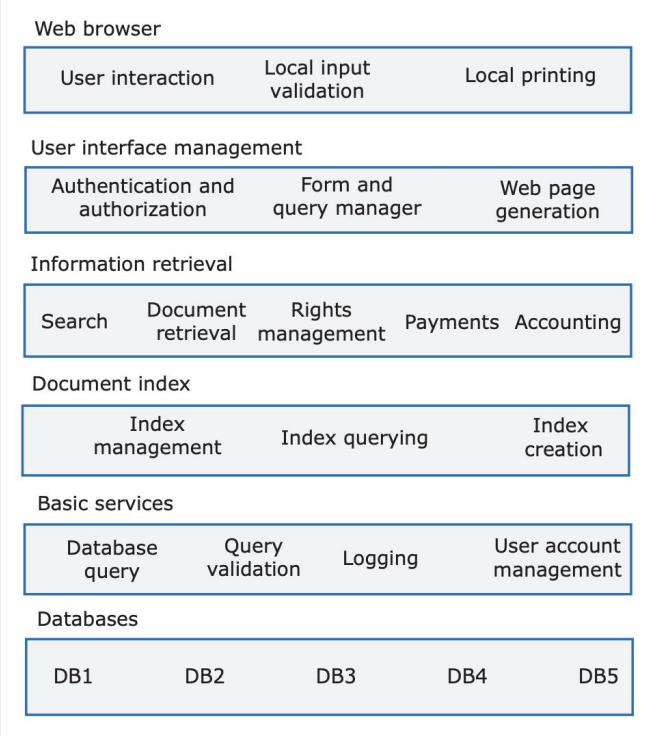
A **service** is a **coherent fragment of functionality**

It can range from a large-scale service (e.g. database) to a **microservice**, which does one **very specific task**.

Services can be accessed directly or through a component API



Software Components: Component, Service and Microservice - Example



Why component decomposition?

Component can be:

- **developed in parallel,**
- **reused**
- **replaced**
- **distributed**

Layers

Layers are **logical grouping of components**.

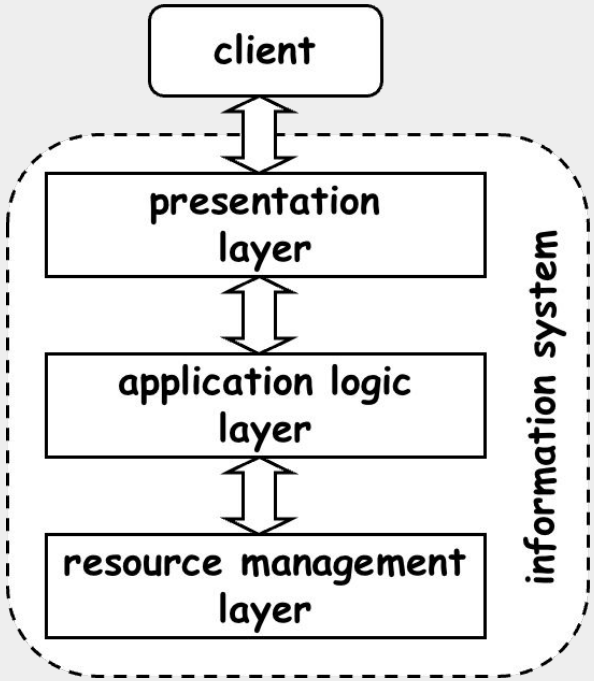
Many software product have a common layered structure

Presentation: manage the interactions with the system

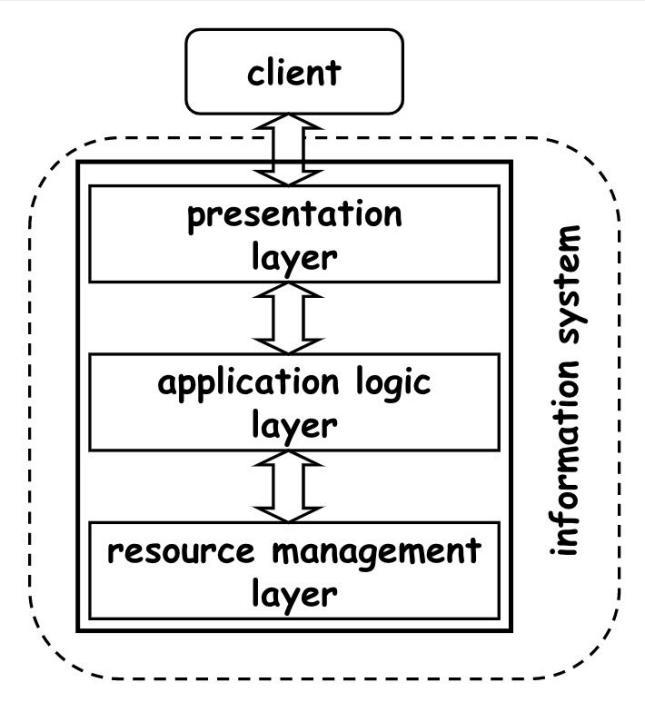
Application: what the system does

Resource management: deals with the organization of the data

Tier: software module implementing a layer



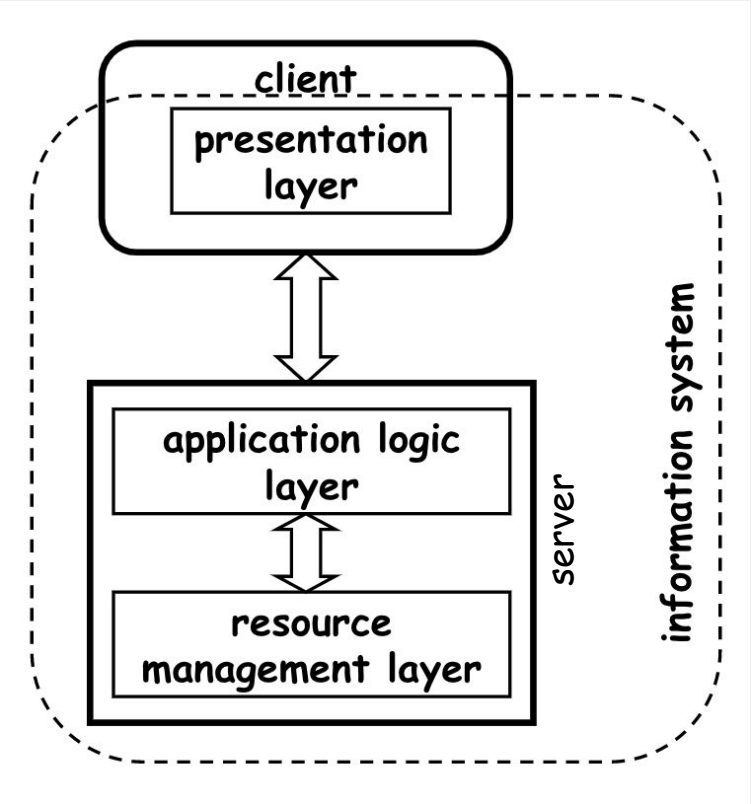
1-Tier Architecture



all is centralized (single machine)

- low portability
- low maintainability
- harder to update
- no “communication costs”
- easier resources management
- layers can be merged ⇒ better performances

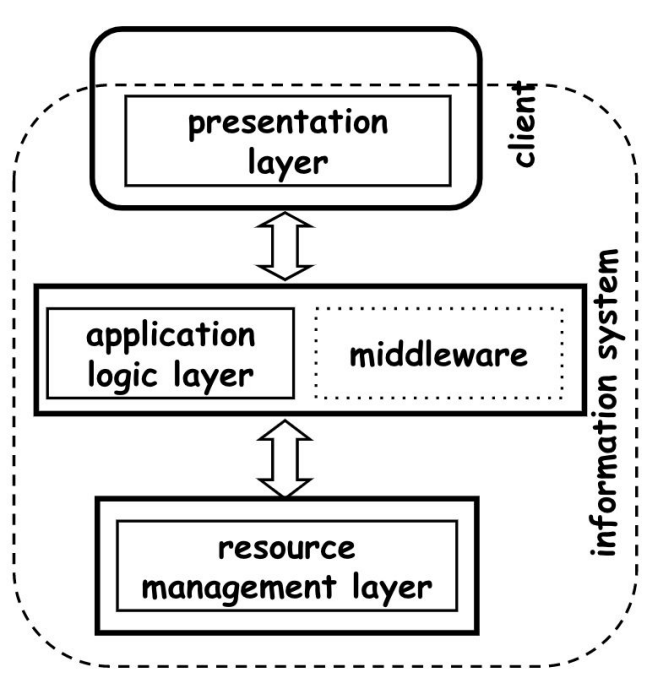
2-Tiers Architecture



Clients are independent of each other: one could have several presentation layers depending on what each client wants to do
⇒ server saves resources

It introduces the concept of **API** (Application Program Interface), an interface to invoke the system from the outside
⇒ it enables integrations

3-Tiers Architecture

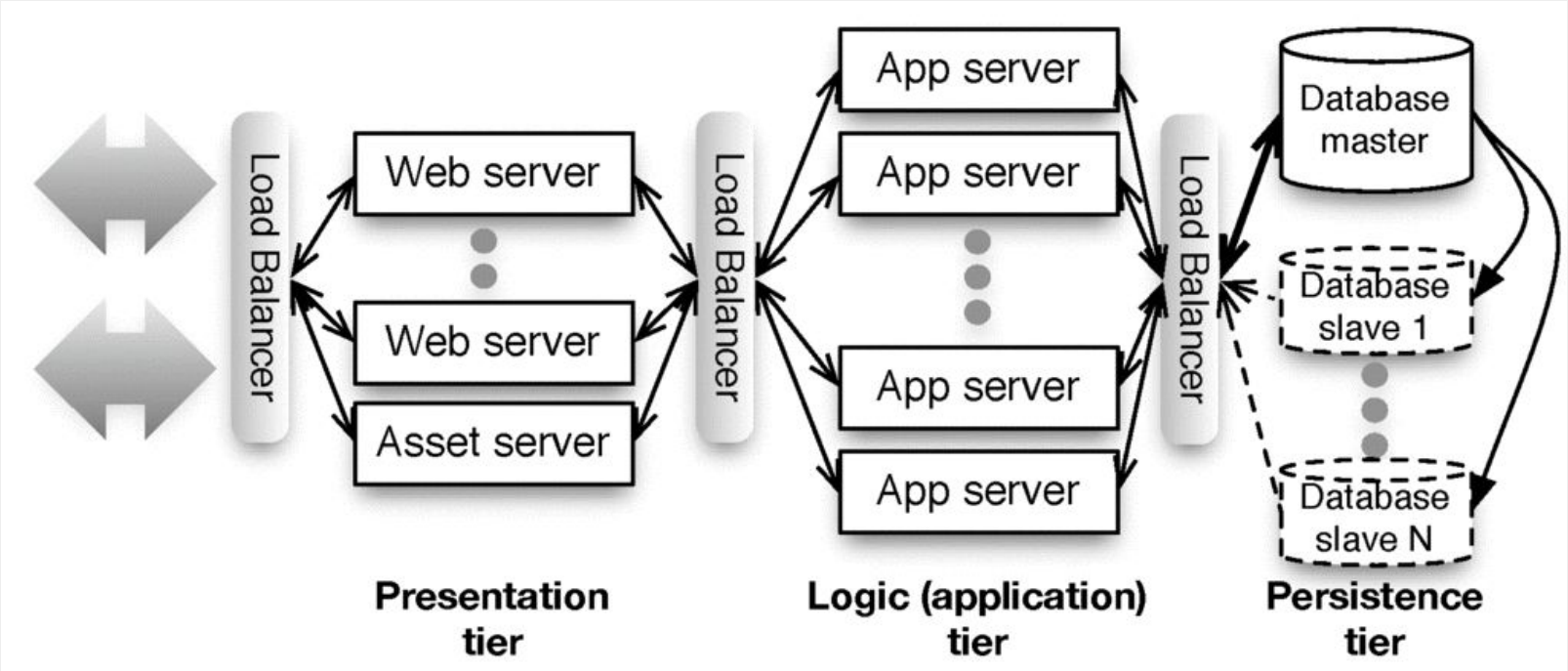


Enables scalability: more servers

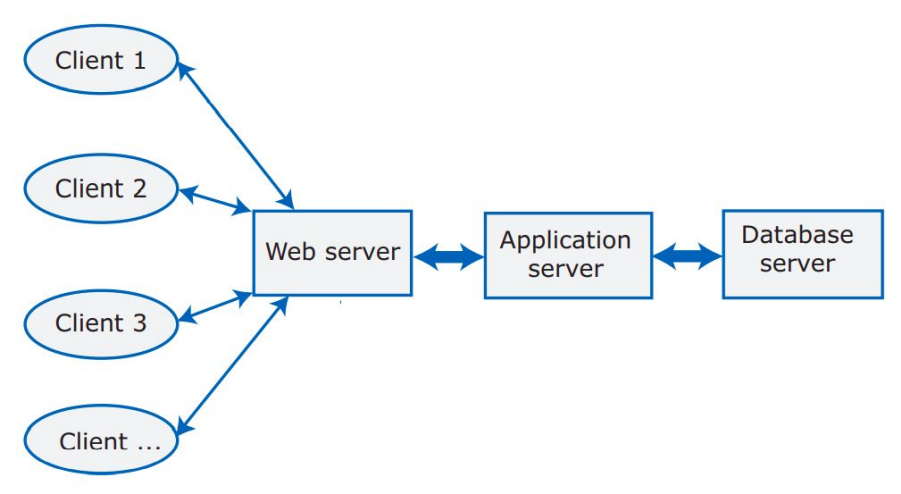
Middleware: abstractions and infrastructures supporting the application layer

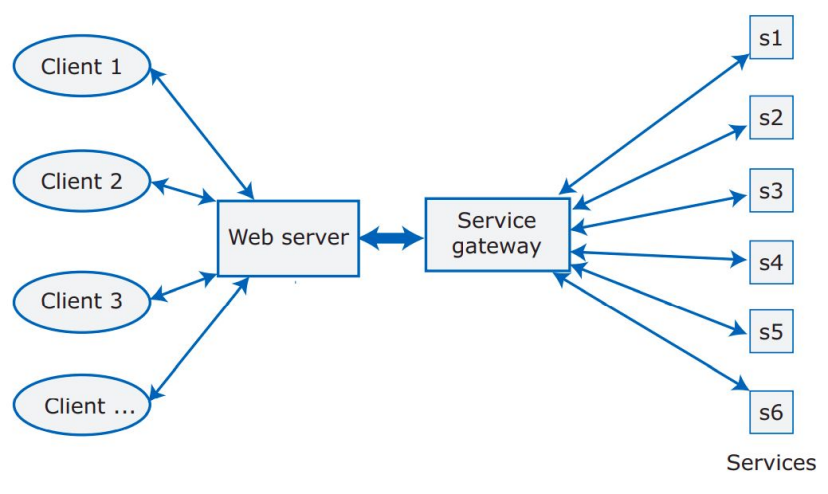
+ more maintainability, easier to update, **more scalability**, can be distributed on different servers
- Communication costs between tiers

3-Tiers Architecture, Scalability



Distributed Architectures : Multi-tier and Service-oriented



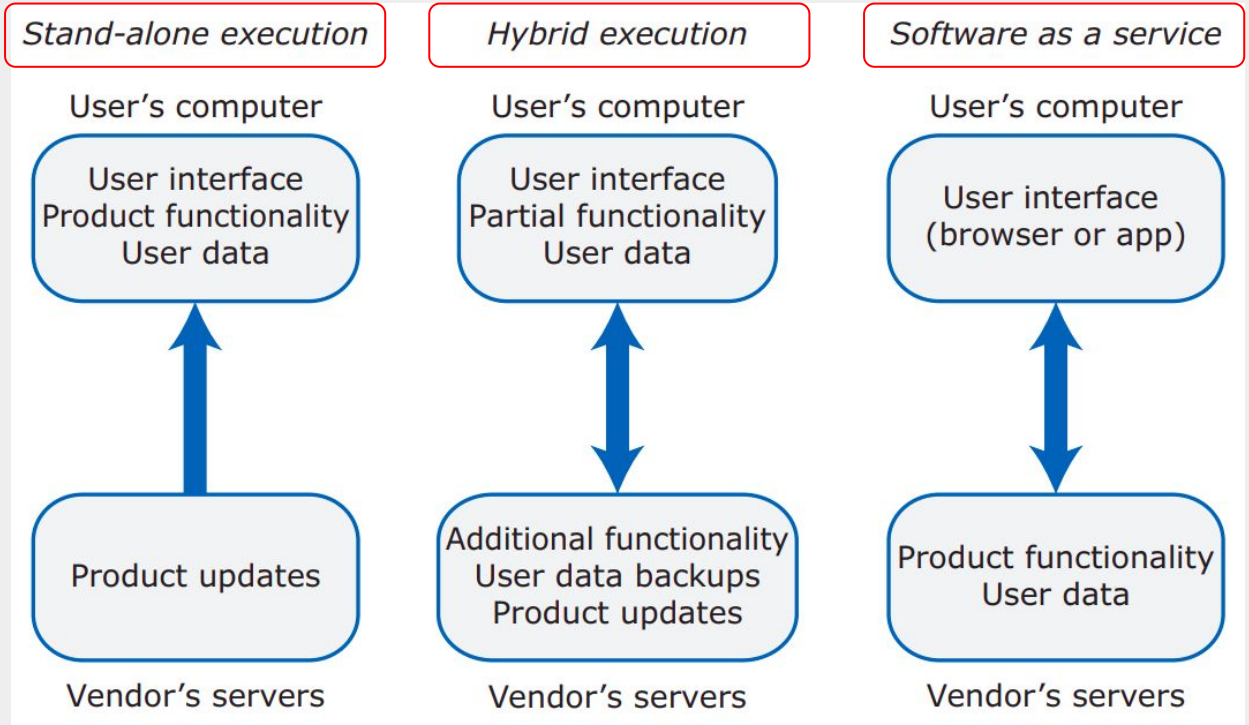


The choice depends on:

- Data type and data updates
- Frequency of change
- System execution platform

Services are stateless components => Easier to scale, update

Software Execution Models



SaaS Advantages

1. **No local installation**, removes hardware compatibility requirement
2. Associated service **data is kept within service** (no back up, data loss concerns)
3. **Easier** for groups of users to **collaborate** on same data
4. Large, frequently updated data, accessed remotely
5. Developers maintain **single copy of server software** (instead of 1 per OS)
6. **Upgrades** to software don't interfere with user experience

Disadvantages?

Cloud

It is a large number of remote **servers** that are **offered for rent**

Remote servers are “virtual” = implemented in software rather than hardware

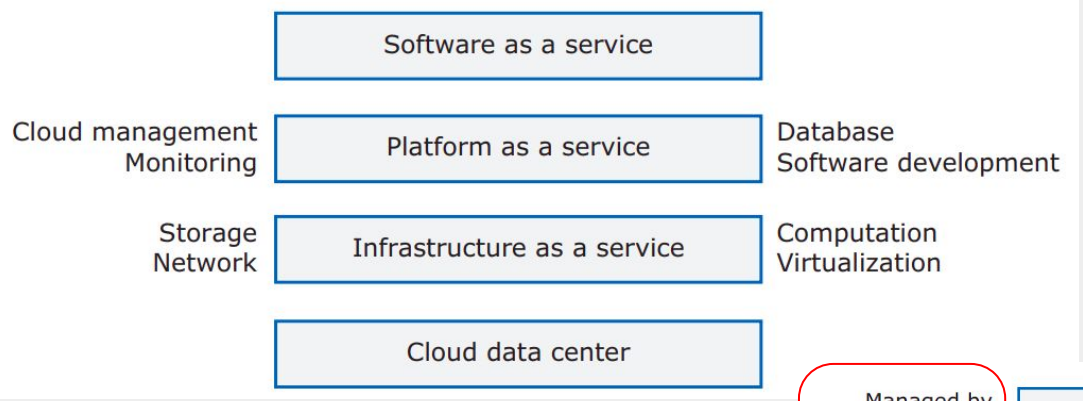
Cloud software features (some): **scalability, elasticity and resilience**

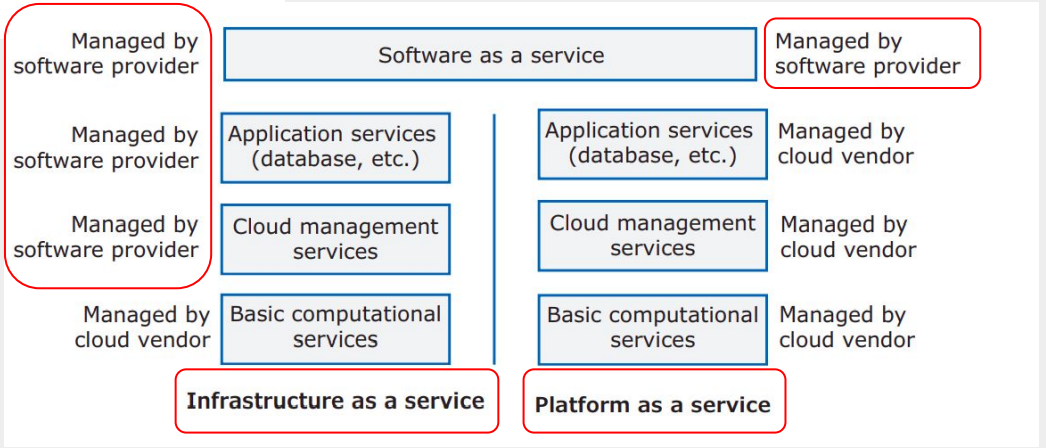
Infrastructure as a Service (IaaS)

Platform as a Service (Paas)

Software as a Service (Saas)

Cloud


Software as a service
Cloud management
Monitoring
Platform as a service
Database
Software development
Storage
Network
Infrastructure as a service
Computation
Virtualization
Cloud data center


Managed by software provider
Software as a service
Managed by software provider
Managed by software provider
Application services (database, etc.)
Application services (database, etc.)
Managed by cloud vendor
Managed by software provider
Cloud management services
Cloud management services
Managed by cloud vendor
Managed by cloud vendor
Basic computational services
Basic computational services
Managed by cloud vendor
Infrastructure as a service
Platform as a service

Architectural Design Guidelines